mTPC Digitisation

What does it do:

Further below are some slides explaining the set up, which might help with the text below.

The mTPC is the recoil detector surrounding the target. It is segmented into 10 separated chambers along the beam line. Low momentum protons pass through the chambers and leave tracks. The chambers are filled with gas. Ionisation is created by the protons, and drifted to the readout planes by an E-field (which is set up by the red HV disks). The readout planes consist of a double layer of GEM foils, then a segmented pad readout.

For the digitisation – g4sbs tracks the protons in the gas volume. At the moment we artificially force the stepping action to get the edep from the proton at very fine steps as it is tracked through the gas volume of the mTPC. This is because g4 is not ideal for modelling energy deposition of low momentum hadrons in a gas, and without this forcing of the stepping action there were hardly any hits which does not seem realistic from experience with other experiments, e.g. BONUS.

At each step/hit in the gas with an energy deposit, the delta electron ionisation yield expected is calculated. This is performed using values obtained from a separate Garfield++ simulation, which was used to find the ionisation energy of the gas, the ionisation drift properties of the gas and the diffusion properties of the gas. Similarly Garfield++ was used to find these numbers for inside the GEM foils too. The digitisation uses these numbers to get the final hits on the segmented readout pad. The “ionisation” is drifted artificially in the digitisation script from the location of the proton hit in the gas to the readout pads. So the g4sbs output gives the proton tracks in the gas, and the digitisation script simulates the ionisation part and implements the readout pad geometry.

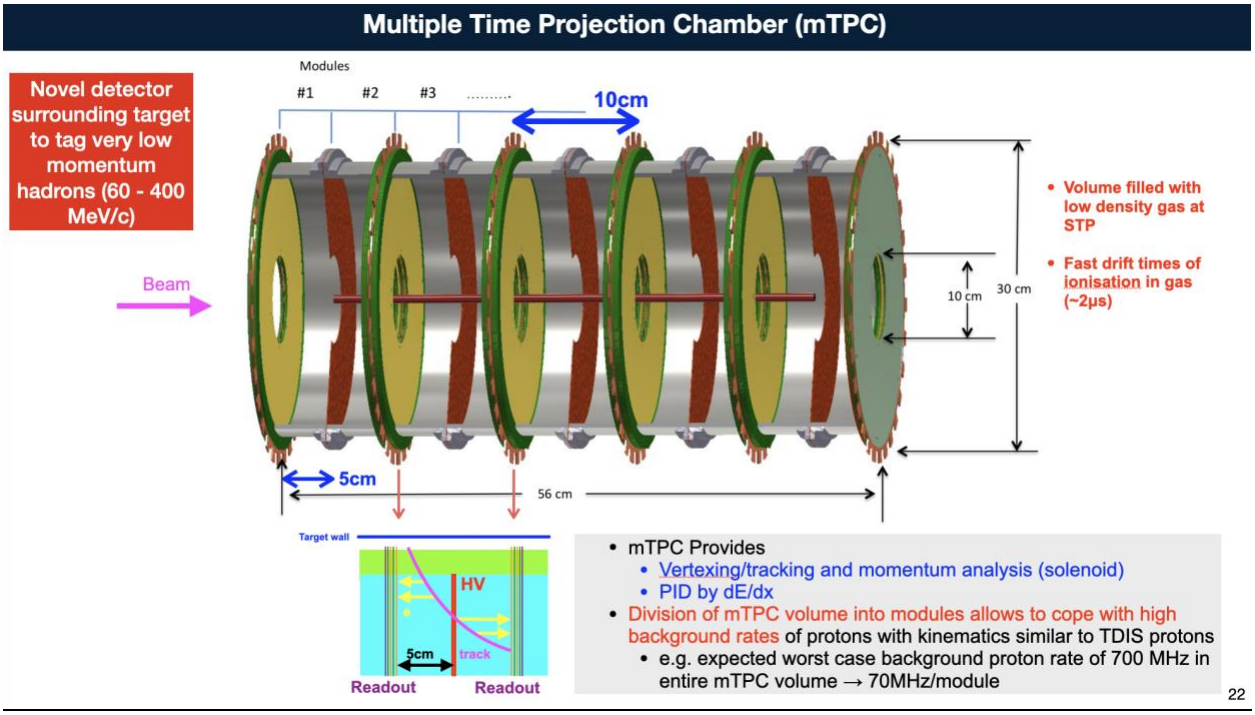
There is a script to generate the readout pad geometry – at the moment it is based on Steve Wood geometry and the script just generates a TH2 with varying bin shapes. The TH2 is saved as a root file and that is read in by the digi script.

There are two methods of digitisation.

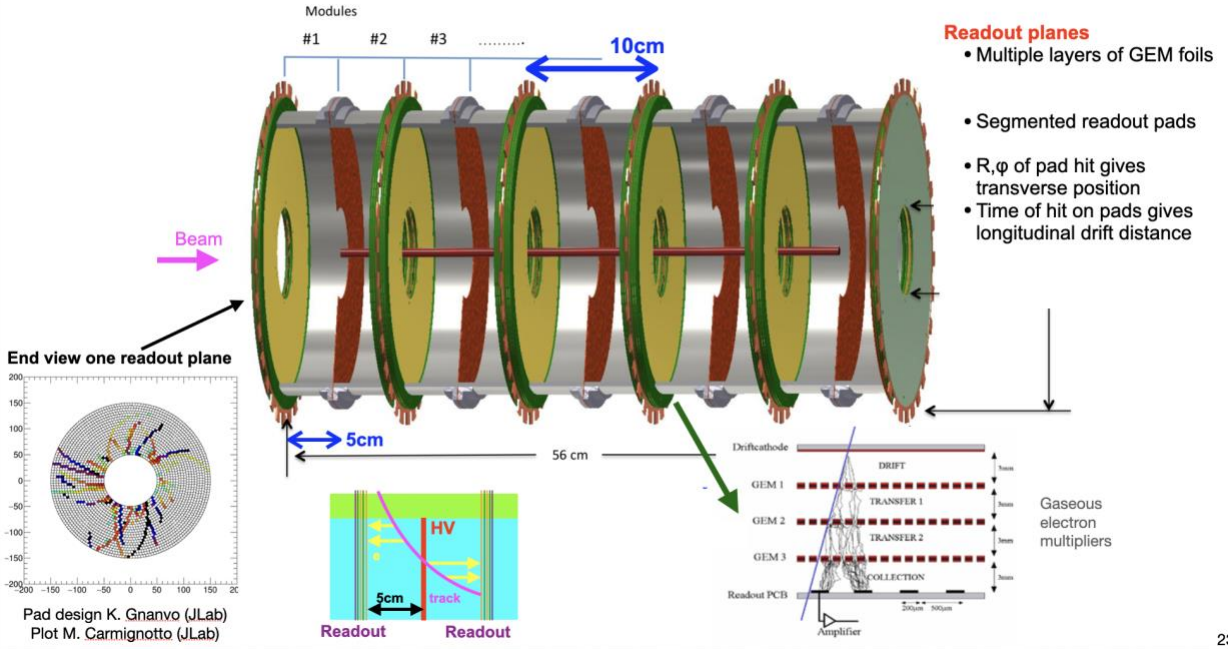
A **simple** method which purely uses a combination of the z drift distance (along the beamline direction) between the hit of the proton in the gas and the position of the readout plane, combined with the drift velocity of the gas, to project which readout pad will have a signal with which time stamp.

A **charge sharing** method which includes transverse (affecting spatial position) and longitudinal (affecting time distribution of the charge arrival on the pads) diffusion of the ionisation charge cloud in both the gas volume and in each layer of gem foils.

Future upgrade I recommend: adding in the readout electronic shaping (SAMPA electronics). I did this in the past, but removed it for simplicity for the background tracking studies. However I have included the macro to simulate the electronics (**SampaShapingAndSample_ReturnVals.cpp**) in the folder. The output of the scripts is the pulse shapes in the last slide below, one would just need to call the function at the end of the digitisation script and take the time stamp eg of the peak of the shaped SAMPA pulse. This could be used to study what eventual time resolution we have and with what background rate we could handle pile up etc. There might be a script somewhere on my home area where I did this already – feel free to search.



Multiple Time Projection Chamber (mTPC)

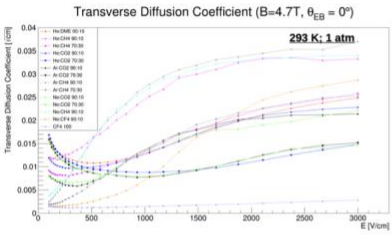


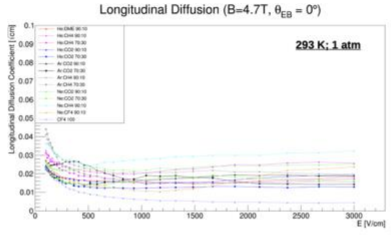
Readout planes

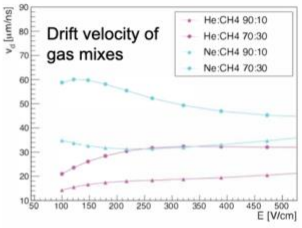
- Multiple layers of GEM foils
- Segmented readout pads
- R,φ of pad hit gives transverse position
- Time of hit on pads gives longitudinal drift distance

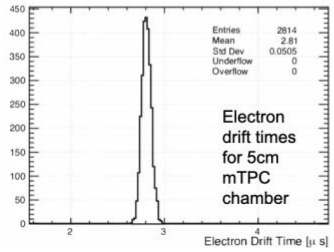
mTPC Simulations

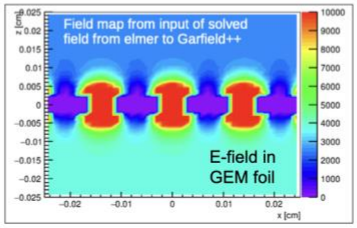
- TPC gas mixture properties and ionisation drift in gas under E and B fields:
  - Need low density gas, fast drift time due to high rates, low diffusion for tracking/momentum resolution
  - Simulated using magboltz and CERN's Garfield++

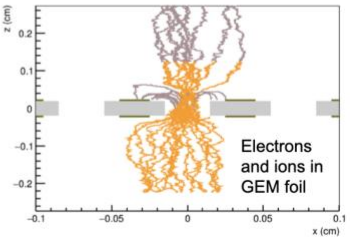


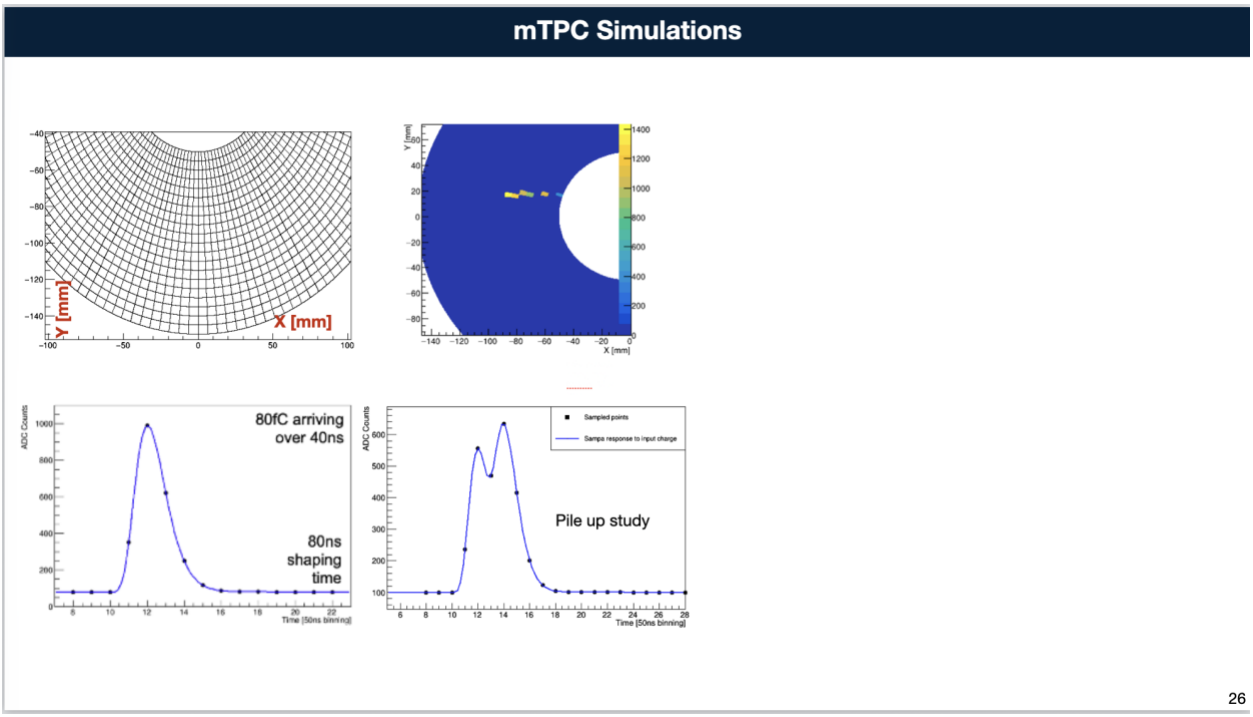












Input Files for Digitisation:

For digitisation in general we have been looking at background hits from the EPC generator which uses the Wiser model.

The input EPC files I have been using originally came from Steve Wood and are at location:

/w/work0/home/rachel/TDIS/EPC

e.g. the file below is one to use as input to g4sbs (can run for less events than 10M in g4sbs by setting the number of events in the mac file for running g4sbs).

/w/work0/home/rachel/HallA/TDIS/EventsFromSteve/mtpcsimepc_10000000forg4_tree.root

Originally the data from Steve was in a text file and was then converted into a root file for input to g4sbs.

The digitisation chain is in general:

- run g4sbs with EPC input file, and the g4sbs settings needed (eg solenoid strength), this generates a typical g4sbs root output file

- Split up the root output file into smaller root files
- Then run the digitisation code over the smaller output root files on the farm
- The output of the digitisation code is then several txt files which are passed on to the team for tracking studies.

There are two formats of the txt files. Some people use “XYZ” hit info (ie they want the truth xyz point for the primary particle hits in the detector written out in the txt file, and some people want “No XYZ” hit info, without the truth info. It’s explained below how to do that when relevant below. At the moment it’s a minor change by hand. Probably could be better to automise it with a flag in the function call of the macro.

Making a readout plane (if needed)

The macro **MakeStaggeredPads.C** does this.

I made the macro to match Steve’s pad design. Could be altered if ever needed different pad designs (but don’t expect this to come up).

It creates hPads2.root which the digitisation script needs.

The root file is too big for github – you will need to take it from my home area at /home/rachel/geant4/TDIS_digi/CheckChargeShare/ForGitHub

(much like what is done with BB and SBS mag field files for g4sbs).

Running g4sbs step

If needed, to generate g4sbs output files for performing digitisation, then need to run correct TDIS branch of g4sbs. The latest branch is tdis_dev.

https://github.com/JeffersonLab/g4sbs/tree/tdis_dev

To create g4sbs output files needed for running with the digitisation code, need to run g4sbs with full particle history mode enabled for the mTPC sensitive detector volume.

Example of scripts for running g4sbs in the correct mode are on npc11 at:

/scratch/TDIS_git/g4sbs/scripts and e.g the input macro batch_tdisAcquRach.mac

Note, in the .mac file, for the digi process it is necessary to run g4sbs in mode AcquMC (a hack put in to read the EPC files – I think since I put this hack g4sbs has been upgraded to take input root files, but would stick with the AcquMC for tdis).
/g4sbs/kine AcquMC

Something to always check in the mac file is the magnetic field strength of the solenoid, just modelled by a uniform solenoid:

```
/g4sbs/solunifield        true

/g4sbs/solunimag          2.8 tesla
```

4.7T is nominal strength, but 2.8T is under study in some cases.

Particle history for mTPC needs to be enabled:

```
/g4sbs/keephistory SBS/mTPC 1
```

Usually I have left the target as D2, unless explicitly asked to change, as that has the worst background compared to H2.

I made the permissions of the g4sbs directory on npc11 open, so perhaps one could even run it directly there without needing to install g4sbs etc.

Splitting g4sbs output file

Macro split2.C.

I usually just split the files into 10k events per file (takes about 9 hours to run on the farm like that).

Need to edit the location of the input and output files by hand in this file at lines 3 and 21 respectively.

The input file is the g4sbs output.

In root, .L split2.C, then split2().

This will divide the g4sbs output into several files with 10k events each and store at the location you specify in line 21.

The digi script itself

Two files.

The simple one is: DigiSimpleFarm.cpp.

Things to note:

- At line 41 it hard coded to read 10k event g4sbs output root files (would be good upgrade to make this not hard coded. Need to change if change the stats per split g4sbs output file).
- At line 131 is the temp location on the farm node where the input file is stored, should match the farm submission script.
- At line 165, again the temp location for the output to be stored on the farm node and should match farm submission script.
- Lines 254 to lines 271 – this is where to modify the output of the text file to contain or not contain XYZ truth info or not. ACTS tracking needs XYZ truth info, GNN also needs XYZ truth info, Steve is the only one who does not need truth info for his standalone tracking studies. I've left the default as having the xyz truth info included. Just comment out one block and uncomment the other if needed to change.

The charge sharing script, which should be used as normal digi script, is FarmDigiChargeShare.cpp.

Things to note:

- Line 36 again hard coded for 10k event output files from g4sbs. Would need to change if running diff no of events per file.
- Line 144 hard coded location of temp input file on farm node, again should match farm submission scripts
- Line 170 hard coded location of temp output file on farm node, again should match farm submission scripts
- Line 483 to 505, again this is where to choose wether to write out truth xyz info or not. Default I have left it at is to write it out.

Running the digi on the Glasgow farm.

Usual batch mode.

Relevant scripts:

CreateRunList.sh. Use this to generate RunList.txt which is the argument for submitting the jobs to the farms and lists the files needed for the eventual stats in increments of 10k events. To execute ./CreateRunList.sh. Then you will have the file RunList.txt.

FarmDigi. Calls the digitisation script and sets paths during farm submission.

At line 13 update the path to be the working directory for submitting the farm jobs where the digi code lies.

Change the input location hosting the split up g4sbs output files at line 16. Then change the desired output location of the digitisation at line 17.

At lines 37 to 39 comment as appropriate to make selection between using the simple digi or charge sharing digi. **Charge sharing digi should be the default.**

submitdigi.sh. Calls and runs FarmDigi, uses RunList.txt as command line argument. At line 12 need to set the working directory for the digi code and farm submission files. Line 15 is the temp dir location on the farm nodes – remember this should match with that in the digitisation scripts themselves.

Execute ./submitdigi.sh RunList.txt to launch the jobs on the farm.

<u>To test farming part if needed:</u>

A 10k event g4sbs output file if needed for testing (to avoid running g4sbs) is at

/w/work0/home/rachel/TDIS/EPC/Feb26-2p8T/g4sbsout

Above macros are at

/home/rachel/geant4/TDIS_digi/CheckChargeShare/ForGitHub

And all files are uploaded to github at:

https://github.com/rachel-m/mTPC-Digitisation

Note hPads2.root is not on github and needs taken from my home ares.

Other notes:

Feel free to improve any aspect of this code!

My to do if I had time:

- Update the farming part of the script to be jlab farm compatible.
- Add back in the electronics shaping function from SampaShapingAndSample_ReturnVals.cpp to include shaping by the readout electronics and enable us to study the min time resolution needed to avoid pile up

- Somehow merge this sw with libsbsdig, to make it less stand alone and more embedded within the sbs sw framework